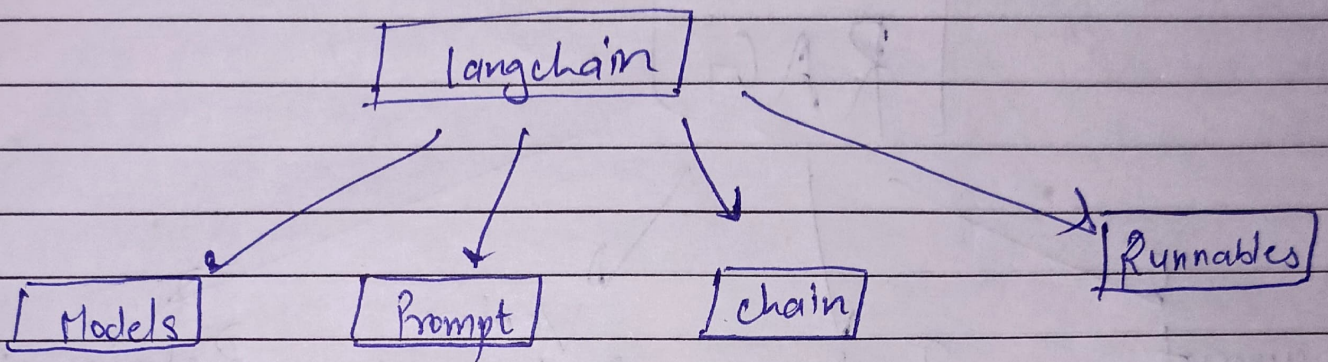
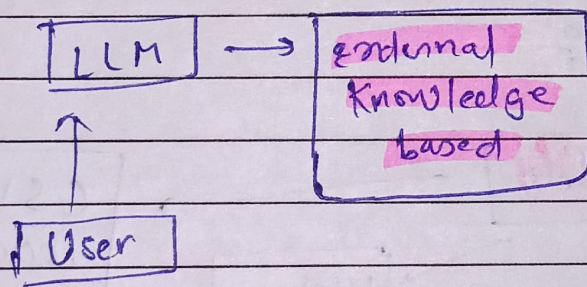


Document loaders in langchain %

Plan of Action



• RAG → (Retrieval Augmented Generation)

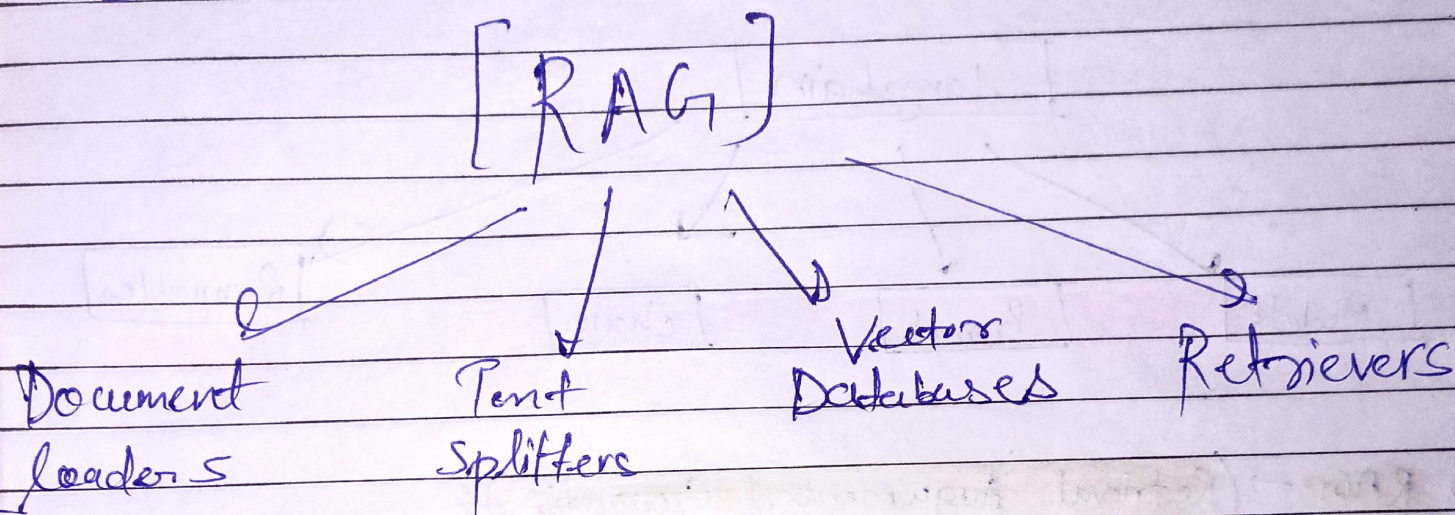


- RAG is a technique that combines information retrieval with language generations, where a model retrieves relevant documents from a knowledge base and then uses them as content to generate accurate and grounded responses.

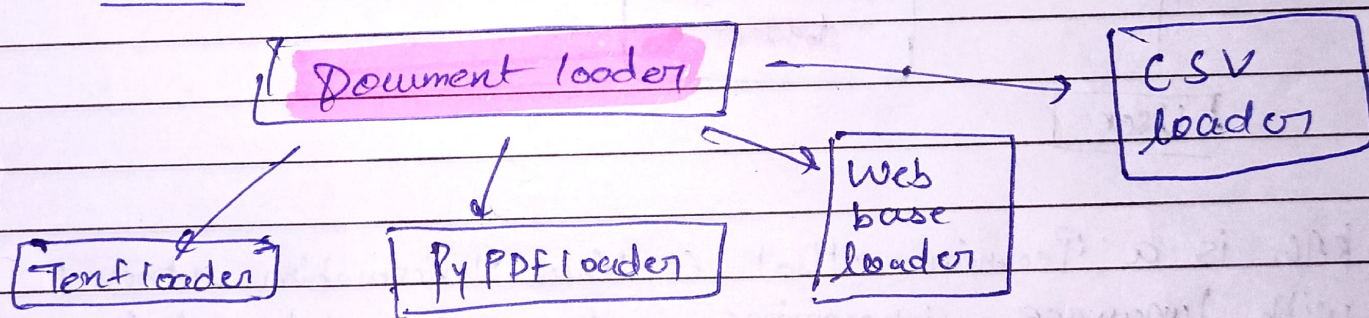
Benefits of using RAG %

1. Use of up to date information
2. Better privacy
3. No limit of documents size.

RAG COMPONENTS



① Document loaders



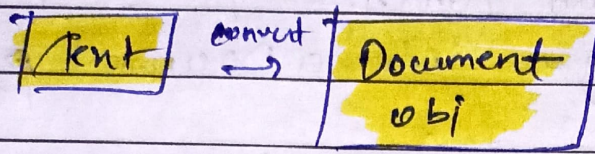
Definition → Document loader are components in langchain used to load data from various sources into a standardized format (usually as document object) which can then be used for chunking embeddings, retrieval and generation.

Document (

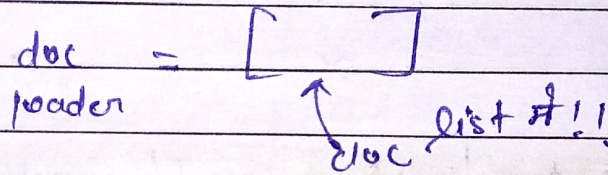
Page-Content = "the actual text Content",

metadata = { "Source" : "file name .pdf", ... }

- Tentloader $\therefore$ is a simple and commonly used document loader in langchain that reads plain tent (.tnt) files and convert them into a langchain document object.



- Ideal for loading chat logs, code snippets, scraped tent, plain text data into a langchain pipelines.
- work only on Tent files (.Tnt) //



from langchain_community.document_loaders import Tentloader

loader = Tentloader(^{.tnt} "file path"; encoding = 'Utf-8')

docs = loader.load()

print = (type(docs)) $\rightarrow$ Type - list

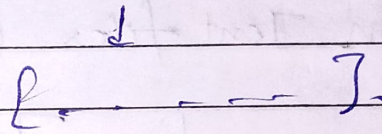
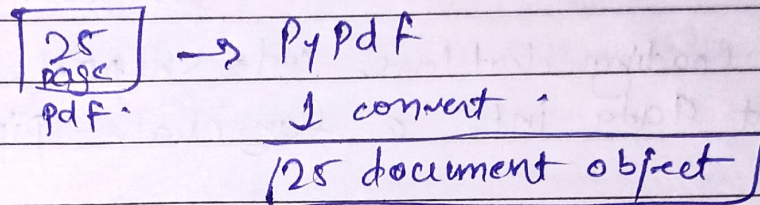
print = (len(docs)) $\rightarrow$ 1 $\rightarrow$ Kitne document load Kiye hai

print = (docs[0]) $\rightarrow$ ^{Prova} Document object print Karega

print = (docs[0].page_content) $\rightarrow$ ~~act~~ actual tent

print = (docs[0].metadata) $\rightarrow$ Extra info.

② PyPDF loader is a document loader in langchain used to load content from Pdf files and convert each pages into a Document object



Document (page-content = 'text from page 4', metadata = { "page": 0, "Source": "file.pdf" }, ...)

Limitations - it uses the PyPDF library under the hood.
- not great with Scanned PDFs or Complex layouts.

PyPDF loader → not good working with Scanned PDF

Use Cases

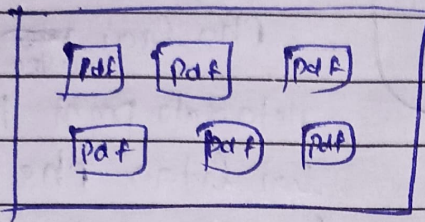
Simpler, clean Pdf
PDFs with Tables / columns
Scanned / Image PDFs
Need layout And Image data
Want best Structure
Extraction

Recommended loaders

PyPDF loader
PDF Plumber Loader
Unstructured PDF loader or Amazon PDF loader
PyMuPDF loader
Unstructured PDF loader

③ Directory Loader → load multiple documents from a directory of files.

Folder Mai Pdf



Glob pattern

What it loads

"**/*.txt"

All .txt files in all subfolders

"*.Pdf"

All .Pdf files in the root directory

"data/*.csv"

All .csv files in the data / folder

"**/*"

All files (any type, all folder)

[** = recursive search through subfolders]

④ Load v/s lazy load ?

load

↓
eager loading

- A list of document obj
- load all ~~memory~~ documents immediately into memory
- Best when
 - the number of documents is small.
 - You want everything loaded up front.

lazy load

↳ generator of docs

- Document are not all loaded at once, they fetched one at a time as needed.
- Best when:
 - you are dealing with large documents or lots of files.
 - you want to stream processing. (eg. chunking, embedding) without

lazy-load :

Date

Note :

docs = loader . lazy-load ()

for document in docs :

print (document.metadata)

Har bar memory mai
ek document aa
rha hai ~~uska~~ ^{uska} hum
Metadata print kr rhe
hai kitna rhe hai
fir agla wala aa rha
hai uska Metadata
print kr rhe hai aur
aisa hi hoga !!

(5) Web Base loader : is a document loader in
langchain used to load and
extract text content from
web pages (URLs).

Request
or

- It uses Beautiful soup under the hood to parse HTML and extract visible text.
- When to use → for blogs, news article, or public websites, where text based content.
- Limitations :
 - Doesn't handle JavaScript heavy pages well (Use Selenium URL Loader for that)
 - Load only static content (what is in the HTML not what loads after the page renders).

- ⑥ CSV_loader → is a document loader used to load
• CSV files into langchain Documents
objects - one per row, by
default.

Same code on Github - - - - -

x x x x x